

# NEXUS ATLAS

Plataforma avançada de inteligência geoespacial  
com IA, satélites e análise tática em tempo real

| Componente          | Tecnologia / Provedor                                   |
|---------------------|---------------------------------------------------------|
| Backend             | FastAPI + SQLAlchemy 2.0 + asyncpg                      |
| Frontend            | React 18 + Vite + TypeScript + Tailwind + Framer Motion |
| Banco de dados      | PostgreSQL 16 (Neon serverless)                         |
| Imagens de satélite | Google Earth Engine (Sentinel-1 SAR + Sentinel-2)       |
| IA (LLM)            | Groq Llama 3.3 70B Versatile                            |
| Visão computacional | YOLOv8 (Ultralytics) + OpenCV                           |
| Mapas               | Mapbox GL JS (3D)                                       |
| Tempo real          | WebSocket (FastAPI + wsproto)                           |
| Pagamentos          | Stripe Checkout + Webhook                               |
| Containers          | Docker multi-stage                                      |
| Deploy              | Render + Neon (configurado: Fly, AWS, Helm)             |
| CI/CD               | GitHub Actions (CI + multi-target deploy)               |
| Repositório         | github.com/Devgabriel01/nexus-atlas                     |

# 1. Visão geral do projeto

---

**NEXUS ATLAS** é uma plataforma SaaS de inteligência geoespacial que combina imagens de satélite reais (Sentinel-1 e Sentinel-2), análise por IA (LLM Llama 3.3 70B + visão computacional YOLOv8), dashboard 3D estilo cyberpunk, multi-tenancy com billing, e pipeline em tempo real via WebSocket.

A arquitetura é inspirada em sistemas militares de OSINT, centros de comando futuristas e em assistentes virtuais como Jarvis. O objetivo é fornecer interpretação tática de mudanças geográficas, detecção automática de anomalias (estruturas, navios, desmatamento) e geração de relatórios em PDF/JSON.

## 2. Backend — FastAPI (Python 3.11)

---

### *O que é*

Framework Python assíncrono moderno baseado em Starlette + Pydantic. Usado para construir a API REST e endpoints WebSocket.

### *Por que usamos*

- Performance comparável a Node.js
- Documentação OpenAPI/Swagger automática em /docs
- Validação de tipos via Pydantic (zero boilerplate)
- Async nativo — essencial pra Earth Engine + Groq + DB simultâneos

### *Onde aparece no projeto*

backend/app/main.py → entry point; backend/app/routers/ → 11 routers (auth, scan, anomalies, reports, exploration, history, analyze, integrations, ws, organizations, billing); backend/app/middleware/rate\_limit.py → rate limiting in-memory.

### *Como funciona — passo a passo*

- 1 Uvicorn (servidor ASGI) recebe requisição HTTP
- 2 CORS middleware valida origem
- 3 Rate limit middleware checa IP (60 req/min)
- 4 Router específico processa a rota
- 5 Dependency injection injeta sessão DB e usuário JWT
- 6 Pydantic valida payload de entrada
- 7 Endpoint executa lógica → serializa resposta via Pydantic

## 3. Banco de dados — SQLAlchemy 2.0 + PostgreSQL (Neon)

---

**SQLAlchemy** é o ORM Python mais robusto, com sintaxe async-first na versão 2.0. **Neon** é Postgres serverless gerenciado (free tier 0.5 GB com auto-pause).

### Modelos definidos

| Modelo       | Função                                                         |
|--------------|----------------------------------------------------------------|
| User         | Conta de usuário (email, password hash, role, organization_id) |
| Organization | Tenant — possui plano, status assinatura, owner                |
| UsageRecord  | Cada scan/AI query é logado pra cota e billing                 |
| Scan         | Sessão de análise (bbox, datas, tipo, status, anomaly_count)   |
| Anomaly      | Deteção individual (type, severity, lat/lon, confidence)       |
| Report       | Relatório gerado com IA (PDF + JSON exportáveis)               |
| Region       | Áreas pré-cadastradas (Amazônia, Ártico, etc.)                 |

**Driver:** asyncpg — cliente Postgres binário ~3× mais rápido que psycopg2. O database.py detecta SQLite vs Postgres e traduz sslmode=require → ssl=True (parâmetro incompatível com asyncpg).

## 4. Autenticação — JWT + bcrypt

**JWT (JSON Web Token):** token assinado que carrega claims do usuário (sub, exp).

**bcrypt:** algoritmo de hash resistente a brute-force.

**python-jose:** biblioteca de assinatura JWT.

**passlib:** wrapper friendly em cima do bcrypt.

### Fluxo de autenticação

- 1 POST /auth/register com email + senha
- 2 Backend hash a senha com bcrypt (cost factor 12)
- 3 Salva User no Postgres
- 4 POST /auth/login verifica hash, retorna JWT (sub=user\_id, exp=24h)
- 5 Toda requisição autenticada inclui Authorization: Bearer <jwt>
- 6 get\_current\_user() decodifica, busca usuário no DB, injeta no endpoint

## 5. WebSocket — pipeline em tempo real

WebSocket é uma comunicação bidirecional persistente sobre TCP. O backend pode **empurrar** eventos pro cliente sem ele perguntar. Scans demoram 30-60 segundos (download de satélite + AI); sem WS, o frontend teria que fazer polling caro.

### Arquitetura pub/sub

event\_bus.py implementa pub/sub em memória usando asyncio.Queue. Cada scan tem um tópico scan:<id>. Subscribers (WebSocket clients) recebem eventos via async for evt in bus.subscribe(topic).

### Sequência de eventos durante um scan

| Stage     | Progress | Mensagem                          |
|-----------|----------|-----------------------------------|
| started   | 5%       | Pipeline initialized              |
| fetch     | 15-35%   | Fetching Sentinel-2 imagery...    |
| sar       | 45-55%   | Fetching Sentinel-1 SAR data...   |
| ai_detect | 60-75%   | Running YOLO + segmentation       |
| temporal  | 82-92%   | Comparing before/after imagery    |
| complete  | 100%     | Scan complete — N anomalies found |

## 6. Google Earth Engine — Sentinel-1 + Sentinel-2

Plataforma do Google que dá acesso ao catálogo público de satélites (Sentinel, Landsat, MODIS) com API Python + servidores de processamento na nuvem.

### Sentinel-2 — visível + infravermelho

- 1 Backend recebe POST /scan com bbox + datas
- 2 Cria geometria `ee.Geometry.BBox(lon_min, lat_min, lon_max, lat_max)`
- 3 Query `COPERNICUS/S2_SR_HARMONIZED` filtrada por bbox + datas + nuvens <20%
- 4 Seleciona bandas RGB (B4, B3, B2), divide por 10000 (reflectância 0-1)
- 5 `geemap.ee_export_image()` baixa GeoTIFF ~1-3 MB

### Sentinel-1 SAR — radar penetrante

SAR penetra nuvens, fumaça e escuridão — único método pra monitorar a Amazônia ano todo. O algoritmo de detecção de mudanças:

- 1 Filtra `COPERNICUS/S1_GRD` em modo IW, polarização VV
- 2 Divide intervalo em duas metades (before/after)
- 3 Calcula mediana de cada metade
- 4 Log-ratio em dB:  $\text{diff} = \text{after} - \text{before}$
- 5 Máscara: pixels com  $|\text{diff}| > 3$  dB são mudança forte
- 6 Sample 50 pontos aleatórios → cada um vira anomalia
- 7 Exporta GeoTIFF do mapa de mudanças (`sar_changes.tif`)

**Autenticação:** User OAuth via `ee.Authenticate(auth_mode='localhost')`. Token persiste em `~/.config/earthengine/credentials`. Projeto: `project-67cfbe40-3c3a-43bd-98e`.

## 7. IA Generativa — Groq + Llama 3.3 70B

**Groq** é uma empresa que criou um chip LPU (Language Processing Unit) que roda LLMs ~10× mais rápido que GPU. Servem o **Llama 3.3 70B Versatile** da Meta, qualidade comparável a GPT-4o-mini,

gratuitamente no tier de dev.

Vantagens

- Velocidade: ~500 tokens/s vs ~50 da OpenAI
- Gratuito: 14.400 tokens/dia, 30 req/min
- Sem cartão de crédito necessário
- API compatível com OpenAI SDK (drop-in)

Pipeline de geração de relatório

- 1 Backend monta prompt estruturado com scan, anomalias detectadas e threat level
- 2 AsyncGroq.chat.completions.create() com response\_format={'type': 'json\_object'}
- 3 Groq retorna análise tática em ~1s (JSON: headline + analysis + recommendations)
- 4 Backend parseia JSON com fallback regex em caso de output malformatado
- 5 Salva no Report row e expõe via /reports/{id}/download/json

8. Visão Computacional — YOLOv8 + OpenCV

**YOLOv8 (Ultralytics):** detector de objetos state-of-the-art. **OpenCV:** processamento de imagem (change detection, NDVI binarization). Detecta automaticamente estradas, navios, aeronaves, estruturas nas imagens.

Estratégia em 3 camadas (graceful fallback)

| Prioridade | Modelo                                             | Quando                            |
|------------|----------------------------------------------------|-----------------------------------|
| 1          | yolov8-satellite.pt (treinado em RarePlanes/xView) | Existe em ai_models/weights/      |
| 2          | HuggingFace satellite model                        | Env SATELLITE_YOLO_HF_REPO setada |
| 3          | yolov8n.pt COCO (78 classes)                       | Fallback final, accuracy reduzida |

Mapeamento classe → severity

| Classe detectada                | Severity atribuída |
|---------------------------------|--------------------|
| large_aircraft, tanker          | critical           |
| aircraft, helicopter, structure | high               |
| ship, vessel, warehouse         | medium             |
| truck, vehicle, building        | low                |

9. Frontend — React 18 + Vite + TypeScript

**React 18:** framework UI declarativo. **Vite:** bundler 100× mais rápido que Webpack (HMR em ~50 ms). **TypeScript:** superset tipado de JavaScript que pega bugs em compile-time. **Tailwind:** utility-first CSS (dark mode trivial). **Framer Motion:** biblioteca de animações declarativas pro HUD estilo Jarvis. **Zustand:** store global leve.

## 10. Mapas — Mapbox GL JS

Biblioteca JavaScript de mapas vetoriais com WebGL — renderiza 3D no browser. Escolhido por: renderização vetorial (suave em qualquer zoom), 3D nativo (tilt, rotate, pitch), estilo dark customizável, free tier de 50k loads/mês.

Token configurado em `frontend/.env` via `VITE_MAPBOX_TOKEN`. Estilo `mapbox://styles/mapbox/dark-v11` com pitch 45° pra perspectiva táctica.

## 11. Multi-tenancy — Organizations + Plans

Estrutura que permite múltiplos clientes (organizações) coexistirem na mesma instância com dados isolados e cotas separadas. Essencial pra vender SaaS.

### Tiers configurados

| Plano      | Scans/mês | AI/mês | SAR | Preço |
|------------|-----------|--------|-----|-------|
| FREE       | 25        | 100    | ■   | \$0   |
| PRO        | 500       | 5.000  | ■   | \$49  |
| ENTERPRISE | 1M+       | 1M+    | ■   | \$499 |

### Enforcement de cota

- 1 Usuário faz POST /scan
- 2 Endpoint carrega org via `current_user.organization_id`
- 3 `enforce_quota(db, org, kind='scan')` é chamado
- 4 Função conta UsageRecord desde início do mês
- 5 Se `>= limit`, raise HTTP 402 Payment Required + `upgrade_url`
- 6 Senão, `record_usage` cria UsageRecord, scan continua

## 12. Pagamentos — Stripe

Stripe é a plataforma de pagamentos com Checkout hospedado — não precisa salvar cartão na sua aplicação (PCI compliance grátis). Funciona em mock mode se `STRIPE_SECRET_KEY` não estiver setado.

### Fluxo de upgrade

- 1 Usuário clica 'Upgrade to PRO' no dashboard
- 2 Frontend chama POST `/billing/checkout {target_plan: 'pro'}`
- 3 Backend cria Stripe Checkout Session, retorna URL
- 4 Frontend redireciona usuário pra Stripe
- 5 Stripe envia webhook `checkout.session.completed` para `/billing/webhook`
- 6 Backend valida assinatura HMAC, atualiza `org.plan = 'pro'`

### 13. Containerização — Docker

Empacota aplicação + dependências em imagem portátil. Sem 'funciona na minha máquina'.

#### Imagens criadas

| Arquivo                   | Tamanho final                   | Usado em           |
|---------------------------|---------------------------------|--------------------|
| Dockerfile.backend.prod   | ~2 GB (GDAL + PyTorch + OpenCV) | AWS, k8s, Fly.io   |
| Dockerfile.backend.render | ~400 MB (slim, sem GDAL/torch)  | Render free tier   |
| Dockerfile.frontend.prod  | ~50 MB (nginx + dist estática)  | Todos os providers |

### 14. Deploy — Render + Neon

**Render** é uma PaaS — você dá um repo Git, ela builda e hospeda. Setup mais rápido (1 clique via Blueprint), detecta render.yaml automaticamente, free tier para web service + static site.

**Neon** é Postgres serverless. Render removeu Postgres free em out/2024, então usamos Neon (free 0.5 GB, sem cartão, auto-pause economiza compute, performance idêntica a RDS pra cargas pequenas).

#### Outros targets configurados

| Provedor          | Custo estimado   | Status no projeto                    |
|-------------------|------------------|--------------------------------------|
| Render + Neon     | \$0 (free tier)  | Blueprint criado, env vars pendentes |
| Fly.io            | \$0-5/mês        | fly.toml pronto, CLI instalado       |
| Railway           | \$5/mês          | railway.json pronto, CLI instalado   |
| AWS (Terraform)   | ~\$80/mês        | main.tf completo (ECS+RDS+ALB+CF)    |
| Kubernetes (Helm) | Custo do cluster | Chart com HPA e Ingress pronto       |

### 15. CI/CD — GitHub Actions

**ci.yml**: roda em cada push. Linta Python (ruff), testa import, builda frontend, e (no main) buildar+push das imagens Docker pro GitHub Container Registry.

**deploy.yml**: manual via workflow\_dispatch. Deploy pra Fly ou Render via API key.

### 16. Fluxo completo de um scan

Sequência end-to-end desde o clique do usuário até o relatório com IA pronto:

| #  | Ator    | Ação        | Detalhe                                       |
|----|---------|-------------|-----------------------------------------------|
| 1. | USER    | POST /scan  | {bbox, dates, scan_type:'full_sar'}           |
| 2. | BACKEND | Quota check | enforce_quota(org, 'sar') + record_usage(...) |

|     |         |                                |                                                 |
|-----|---------|--------------------------------|-------------------------------------------------|
| 3.  | BACKEND | Persist Scan row               | Status: PENDING → background_task agendado      |
| 4.  | BACKEND | Return 202                     | {scan_id, status: pending}                      |
| 5.  | USER    | WS connect                     | ws://api/ws/scans/<id>?token=jwt                |
| 6.  | WORKER  | Sentinel-2 fetch               | Earth Engine: download GeoTIFF                  |
| 7.  | WORKER  | Sentinel-1 SAR                 | Log-ratio change detection                      |
| 8.  | WORKER  | YOLOv8 detect                  | Bounding boxes + class + confidence             |
| 9.  | WORKER  | Temporal CV                    | OpenCV before/after change                      |
| 10. | WORKER  | Persist anomalies              | Postgres: N rows in Anomaly table               |
| 11. | WS      | Emit complete                  | {stage:'complete', progress:100, anomaly_count} |
| 12. | USER    | POST /reports                  | {scan_id}                                       |
| 13. | BACKEND | Groq Llama 3.3 70B             | Gera headline + analysis + recommendations      |
| 14. | BACKEND | ReportLab → PDF                | Salva em /reports/<report_id>.pdf               |
| 15. | USER    | GET /reports/<id>/download/pdf | FileResponse                                    |

## 17. Estado atual e próximos passos

### ■ O que está pronto

| Componente       | Status         | URL                                 |
|------------------|----------------|-------------------------------------|
| Backend local    | ■ Online       | http://localhost:8000               |
| Frontend local   | ■ Online       | http://localhost:5173               |
| Postgres Neon    | ■ Populado     | sa-east-1 (neon.tech)               |
| GitHub repo      | ■ 7 commits    | github.com/Devgabriel01/nexus-atlas |
| Render Blueprint | ■ Criado       | Aguardando env vars                 |
| Earth Engine     | ■ Autenticado  | project-67cfbe40-3c3a-43bd-98e      |
| Groq AI          | ■ Active       | Llama 3.3 70B Versatile             |
| Mapbox           | ■ Token válido | Verificado HTTP 200                 |

### ■■ Pendências

- Setar DATABASE\_URL, GROQ\_API\_KEY, GEE\_PROJECT no Render backend
- Setar VITE\_MAPBOX\_TOKEN no Render frontend
- Resolver billing lock do GitHub (caso queira usar GH Actions)
- Opcionalmente, treinar YOLOv8 satellite-aware em RarePlanes

# Apêndice A — Resumo de todas as tecnologias usadas

| Camada              | Tecnologia               | Função                           |
|---------------------|--------------------------|----------------------------------|
| Linguagem (backend) | Python 3.11              | Linguagem principal              |
| Web framework       | FastAPI 0.115            | API REST + WebSocket assíncrona  |
| Servidor ASGI       | Uvicorn + Gunicorn       | Produção: 2 workers, healthcheck |
| ORM                 | SQLAlchemy 2.0           | Modelagem + queries async        |
| Driver PG           | asyncpg 0.29             | Cliente Postgres binário         |
| Banco               | PostgreSQL 16 (Neon)     | Storage relacional serverless    |
| Banco dev           | SQLite +aiosqlite        | Desenvolvimento local            |
| Validação           | Pydantic v2              | Schemas + settings               |
| Auth                | python-jose + bcrypt 4   | JWT + hash de senhas             |
| WebSocket           | wsproto + websockets     | Real-time pipeline events        |
| IA / LLM            | Groq SDK + Llama 3.3 70B | Interpretação tática             |
| Visão computacional | Ultralytics YOLOv8       | Deteção de objetos               |
| Image processing    | OpenCV 4.10              | Change detection                 |
| Geospatial          | earthengine-api 0.1.408  | Sentinel-1/2, Landsat            |
| Geospatial helper   | geemap 0.34              | Export GeoTIFF                   |
| Numpy               | 1.26                     | Operações matriciais             |
| Shapely             | 2.0                      | Geometria vetorial               |
| Reports             | ReportLab 4.2            | Geração de PDF                   |
| Pagamentos          | Stripe SDK 10.12         | Checkout + webhooks              |
| Linguagem (front)   | TypeScript 5             | Tipagem estática                 |
| UI framework        | React 18                 | Componentes declarativos         |
| Bundler             | Vite 5                   | Dev server + build               |
| CSS                 | TailwindCSS 3            | Utility-first                    |
| Animações           | Framer Motion            | HUD cyberpunk                    |
| State               | Zustand                  | Store global leve                |
| Mapas               | Mapbox GL JS             | Mapas 3D vetoriais               |
| Container           | Docker multi-stage       | Imagens portáteis                |
| Web server          | nginx alpine             | Servir frontend estático         |
| Orquestração        | docker-compose           | Dev local com Postgres+PostGIS   |
| Cloud (IaC)         | Terraform 1.5+           | Provisioning AWS                 |
| Cloud (k8s)         | Helm chart               | Kubernetes deployment            |

|               |                     |                       |
|---------------|---------------------|-----------------------|
| Versionamento | Git + GitHub        | Source control        |
| CI/CD         | GitHub Actions      | Build + test + deploy |
| Registry      | GHCR                | Container registry    |
| CLIs          | gh, flyctl, railway | Tooling de deploy     |

## Apêndice B — Endpoints da API

| Método | Rota                        | Função                      |
|--------|-----------------------------|-----------------------------|
| POST   | /auth/register              | Cria usuário, retorna JWT   |
| POST   | /auth/login                 | Login, retorna JWT          |
| GET    | /auth/me                    | Dados do usuário atual      |
| POST   | /scan/                      | Inicia scan (cota aplicada) |
| GET    | /scan/                      | Lista scans do usuário      |
| GET    | /scan/{id}                  | Detalhes de um scan         |
| GET    | /scan/{id}/status           | Status + progresso          |
| WS     | /ws/scans/{id}              | Stream tempo real           |
| GET    | /anomalies/                 | Lista anomalias             |
| POST   | /reports/                   | Gera relatório com IA       |
| GET    | /reports/{id}/download/pdf  | Download PDF                |
| GET    | /reports/{id}/download/json | Download JSON               |
| POST   | /analyze/coordinates        | AI readout pra coordenada   |
| POST   | /analyze/ai-query           | Query livre Llama 3.3       |
| POST   | /analyze/ndvi               | Cálculo de NDVI             |
| GET    | /analyze/ai/status          | Status do Groq              |
| POST   | /orgs/                      | Cria organização            |
| GET    | /orgs/me                    | Org + plano + uso           |
| GET    | /orgs/plans                 | Lista FREE/PRO/ENTERPRISE   |
| POST   | /billing/checkout           | Stripe Checkout Session     |
| GET    | /billing/usage              | Counters do mês             |
| POST   | /billing/webhook            | Stripe events handler       |
| GET    | /integrations/status        | Status das integrações      |
| GET    | /health                     | Liveness probe              |
| GET    | /                           | Root info                   |

# NEXUS ATLAS

## Plataforma de Inteligência Geoespacial

---

Este documento descreve a arquitetura completa, escolhas técnicas e estado atual do projeto NEXUS ATLAS — uma plataforma SaaS de análise geoespacial construída com Python, FastAPI, React, Earth Engine, YOLOv8 e Llama 3.3.

[github.com/Devgabriel01/nexus-atlas](https://github.com/Devgabriel01/nexus-atlas)  
Documento técnico v1.0